

Лекция 1. Введение в проектирование баз данных. Данные и модели представления.

Дисциплина: Методы и средства проектирования баз данных

Составитель: Монастырев Егорий Егорьевич, М-ВТ-25

Якутск, 2026

Проектирование баз данных — одна из фундаментальных инженерных компетенций разработчика программного обеспечения: веб-приложения, банковские и научные системы, корпоративное ПО.

Главная идея

Разработчик работает не только с кодом, но и с данными, которые этот код создаёт и использует. Прежде чем спроектировать хранение, необходимо понять природу самих данных.

1. Чем данные отличаются от информации и знаний?
2. Как классифицируются данные по источнику, формату и структуре?
3. Из каких этапов состоит жизненный цикл данных?
4. Чем БД отличается от СУБД?
5. Чем табличное представление отличается от документного?
6. Чем JSON отличается от NoSQL?
7. Что такое объектное хранилище?
8. Что такое ER-диаграмма и из чего она состоит?
9. Что такое архитектура ANSI/SPARC?
10. Зачем нужна нормализация?

В стандартах и терминологических словарях понятие данных связывается с представлением информации в формализованной форме, пригодной для передачи, интерпретации или обработки.

Определение (по терминологии NIST)

Данные — представление фактов, концепций или инструкций в форме, пригодной для коммуникации, интерпретации или обработки человеком либо автоматическими средствами.

Примеры данных:

101 Иванов И.И. 23.7 2026-09-02

Само значение не всегда содержит информацию о том, что именно оно обозначает.

101

может быть:

идентификатор студента номер группы номер заказа код объекта

Вывод

Значение данных должно рассматриваться совместно с контекстом и правилами его интерпретации.

Для учебных целей полезно разграничивать три уровня:

Данные → Информация → Знания

- ▶ **Данные** — формализованные представления фактов, наблюдений, измерений.
- ▶ **Информация** — данные, интерпретированные в определённом контексте и обладающие смыслом для получателя (согласуется с терминологией ISO и NIST).
- ▶ **Знания** — организованная совокупность понятий и связей между ними, позволяющая интерпретировать информацию и использовать её при решении задач.

Термин «знание», в отличие от «данных» и «информации», имеет разные трактовки в литературе; здесь он используется как более высокий уровень организации информации.

Пример: от данных к знаниям

Данные

Студент: 101
Математика: 5
Программирование: 5
Физика: 3

Информация

Студент 101 имеет высокие оценки по математике и программированию и более низкую — по физике.

Знание

Выраженная успеваемость студента в дисциплинах, связанных с математикой и программированием.

Данные → Интерпретация → Информация → Обобщение опыта
→ Знание

Почему важно различать данные и информацию

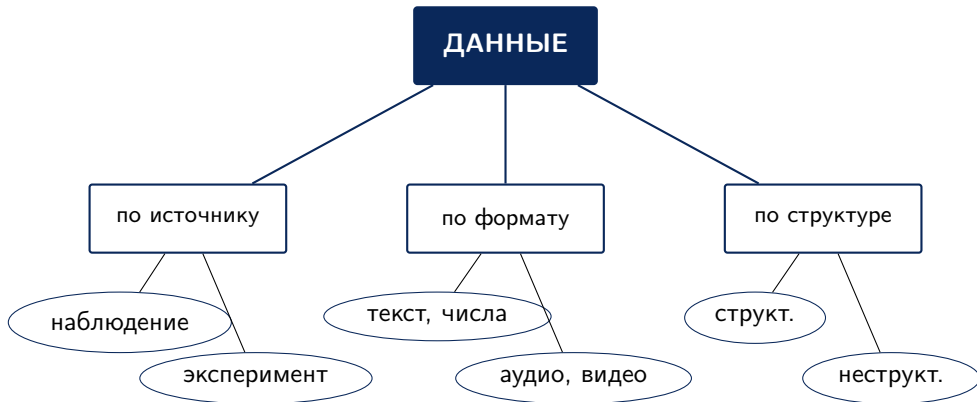
Проектирование информационной системы предполагает работу не просто со значениями, а с их:

смыслом структурой происхождением
взаимосвязями ограничениями временем существования

Ключевой вопрос проектировщика

Не только **что хранить?**

НО И что означает каждый элемент данных?



Дополнительно данные классифицируются по характеру изменения во времени.

Откуда берутся данные

Источник	Примеры
Наблюдения	датчики, опросы, измерения
Эксперименты	результаты лабораторных экспериментов
Моделирование	результаты математических моделей
Результаты обработки	статистика, средние значения, прогнозы
Справочные данные	классификаторы, справочники

Данные → Обработка → Новые данные

В каком виде представлены данные

- ▶ **Текст** — документы, сообщения, описания;
- ▶ **Числа** — измерения, оценки, статистика;
- ▶ **Изображения** — фотографии, схемы, рисунки;
- ▶ **Аудио и видео** — звукозаписи, фильмы, видеозаписи;
- ▶ **Файлы** — PDF, архивы, программы и другие данные.

Важно

Формат показывает, **в каком виде записаны данные**, но не объясняет их смысл.

Тип	Что это значит	Примеры
Структурированные	имеют заранее заданную структуру	таблицы, базы данных
Полуструктурированные	структура есть, но она более гибкая	JSON, XML
Неструктурированные	нет заранее заданной структуры	текст, фото, видео

Главное

Чем сложнее структура данных, тем больше вариантов их хранения и обработки.

Как изменяются данные со временем

Неизменяемые

После создания данные **не меняются**.

Примеры:

архив,
результат эксперимента

Растущие

Старые данные сохраняются, добавляются **новые**.

Примеры:

журнал событий,
история измерений

Изменяемые

Существующие данные могут **изменяться** или удаляться.

Примеры:

профиль пользователя

Почему это важно?

Способ изменения данных напрямую влияет на их хранение, обновление и резервное копирование.

Определение

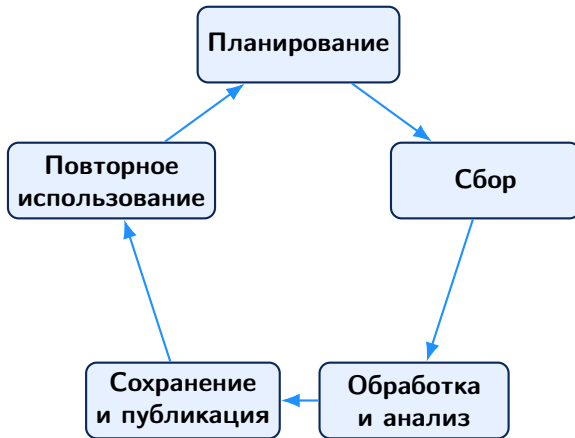
Термин **Big Data** обозначает данные, **объём, разнообразие, скорость поступления** или другие характеристики которых создают существенные требования к традиционным методам их обработки и хранения.

Примеры источников

- ▶ транзакции
- ▶ сенсорные данные
- ▶ изображения
- ▶ журналы событий
- ▶ социальные данные
- ▶ видео

Жизненный цикл данных

Данные существуют не только в момент их записи в базу.



Согласно UK Data Service, жизненный цикл исследовательских данных представляется этапами:
plan → collect → process and analyse → preserve and share → reuse.

Ключевые этапы жизненного цикла

Этап	Суть
Планирование	определяются форматы, место хранения, доступ, требования к защите — <i>до</i> создания данных
Сбор	данные регистрируются, классифицируются, снабжаются метаданными
Обработка и анализ	проверка, очистка, преобразование, агрегация; результат — производные данные
Документирование	сохраняются сведения о происхождении, структуре, методике получения, версиях
Сохранение и доступ	резервные и архивные копии, контроль целостности, управление правами доступа
Повторное использование	хорошо документированные данные используются в новых задачах



- ▶ **Конфиденциальность** — доступ получают только уполномоченные субъекты;
- ▶ **Целостность** — данные не должны изменяться несанкционированным образом;
- ▶ **Доступность** — уполномоченные пользователи должны получать данные тогда, когда это необходимо.

Определение (по терминологии NIST)

База данных рассматривается как репозиторий информации или данных; такое определение не ограничивает базу данных только традиционной реляционной системой.

Основное положение

База данных не тождественна таблице. Таблица — лишь один из возможных способов представления структурированных данных.

Современные информационные системы работают со структурированными данными: интернет-магазин, банковская система, корпоративная CRM, социальная сеть.

Приложение $\longleftrightarrow$ База данных

АИС (автоматизированная информационная система) — совокупность программно-технических средств и персонала, обеспечивающая сбор, хранение, обработку и предоставление информации. БД — важнейший, но не единственный компонент АИС.

Создать таблицы → Добавить столбцы → Записать данные

На небольшом примере такой подход может работать, но при росте системы появляются проблемы:

- ▶ дублирование и противоречивые значения;
- ▶ аномалии вставки, обновления и удаления;
- ▶ сложность изменения структуры;
- ▶ падение производительности запросов.

Главная мысль

Ошибки в структуре данных часто обнаруживаются только после того, как система уже начала работать.

Проектирование БД $\neq$ CREATE TABLE

Проектирование отвечает на вопросы: какие данные хранить, как они связаны, какие ограничения существуют, какие операции будут выполняться и как система будет расти и масштабироваться.

Определение

Проектирование базы данных — процесс определения структуры, взаимосвязей, ограничений и способов хранения данных в соответствии с требованиями информационной системы.

Понятие	Определение
База данных	структурированный (или иначе организованный) репозиторий данных
СУБД	программное обеспечение для создания, хранения и обработки БД

Примеры СУБД:

PostgreSQL Microsoft Access MySQL Oracle Database

Зачем нужна СУБД и что она делает

Просто хранить файлы с данными недостаточно.

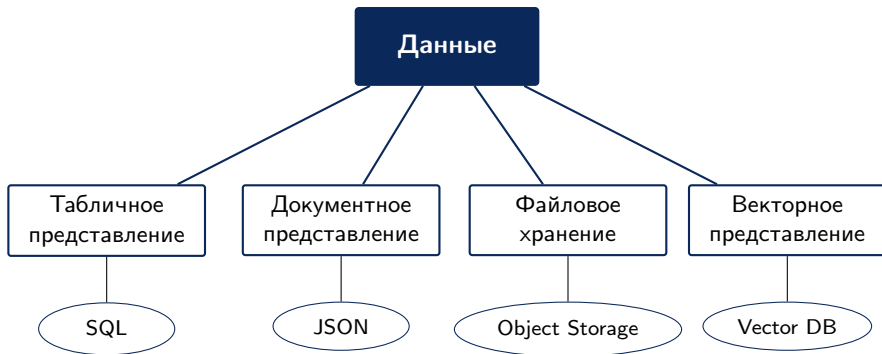
Функция	Суть
Хранение	таблицы, индексы, страницы
Транзакции	атомарное выполнение операций
Журнализация	восстановление после сбоев
Целостность	проверка ограничений
Безопасность	управление доступом и правами
Оптимизация	эффективное выполнение запросов

Метаданные — данные, описывающие структуру, свойства и правила использования других данных («данные о данных»): имена таблиц и столбцов, типы данных, ключи, ограничения, права доступа.

Словарь данных (Data Dictionary) — компонент СУБД, содержащий метаданные о структуре БД.

В PostgreSQL:

`pg_catalog` `information_schema`



Важно различать **модель представления данных** и **физическое средство хранения** — это не одно и то же.

В реляционной модели данные представлены в виде **отношений**; практическим представлением отношения является таблица.

id	name	group_id
1	Иванов	101
2	Петров	101
3	Сидоров	102

Табличная модель характеризуется фиксированной структурой, атрибутами, идентификацией экземпляров, связями и ограничениями целостности.

Определение (RFC 8259)

JSON — лёгкий текстовый, независимый от языка формат обмена данными, предназначенный для переносимого представления структурированных данных.

```
{  
  "id": 101,  
  "name": "Иванов Иван",  
  "group": { "number": "ПМИ-25", "course": 2 },  
  "subjects": ["Математика", "Программирование"]  
}
```

JSON поддерживает объекты, массивы, строки, числа, логические значения и `null`.

NoSQL — широкий класс систем управления данными, не ограниченных классической реляционной моделью. В курсе рассматривается прежде всего документная модель как один из вариантов нереляционного представления.

Принципиально важно

JSON и NoSQL — не синонимы.

JSON — **формат** представления данных.

NoSQL — более широкий **класс подходов** к организации и управлению данными.

При документном подходе совокупность связанных атрибутов объекта представляется единым документом с вложенной структурой.

- ▶ допускает различия в структуре отдельных документов;
- ▶ естественно представляет иерархические данные;
- ▶ приближает представление данных к структуре объектов приложения.

Оборотная сторона

Гибкость структуры переносит часть ответственности за согласованность данных на уровень проектирования и программного обеспечения.

Некоторые объекты удобно описывать не набором отдельных полей, а **вектором признаков** в многомерном пространстве.

$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$

В задачах машинного обучения и поиска вектор может представлять:

- ▶ текстовый документ;
- ▶ изображение;
- ▶ аудиозапись;
- ▶ профиль пользователя;
- ▶ результат работы нейронной сети.

Сравнение способов представления данных

Характеристика	Табличное	Документное	Векторное
Основная единица	строка / отношение	документ	вектор
Структура	явная схема	гибкая / иерархическая	фиксированная размерность
Основная операция	запрос по атрибутам	доступ к документу	поиск по близости
Типичный пример	PostgreSQL	MongoDB	Vector DB

Важно

Это не взаимоисключающие технологии. Одна информационная система может одновременно использовать таблицы, документы, файлы и векторные представления.

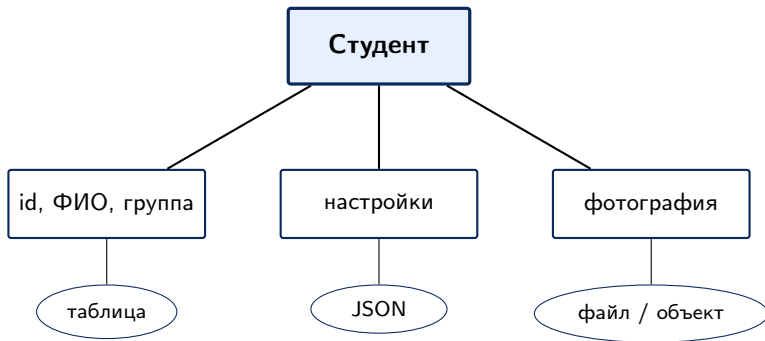
Для больших бинарных объектов (фото, видео, аудио, PDF, архивы) используются файловые или объектные хранилища.

Определение (по документации Amazon S3)

Объект объектного хранилища включает как минимум **содержимое** и **ключ**, по которому он идентифицируется; также могут использоваться **метаданные**.
Объекты размещаются в контейнерах (bucket).

Bucket → Object (Key + Data + Metadata)

Один объект — несколько структур хранения



Вывод

Один объект предметной области может быть представлен несколькими структурами хранения одновременно.

Критерии выбора способа хранения

- ▶ структура данных: фиксированная или изменяемая;
- ▶ вложенность: плоская или иерархическая;
- ▶ объём: KB, MB, GB, TB;
- ▶ частота изменения: редкая или частая;
- ▶ характер доступа: по идентификатору, по атрибутам, по связям, последовательное чтение;
- ▶ требования к целостности: строгие ограничения или более гибкая структура.

Вывод

Место и способ хранения выбираются исходя из свойств данных, а не наоборот.

Предметная область — часть реального мира (фрагмент, рассматриваемый в рамках конкретной задачи моделирования), информацию о которой необходимо хранить и обрабатывать.

университет магазин библиотека больница

Первый шаг проектирования — не **SQL**, а анализ предметной области: какие объекты существуют, какими характеристиками обладают, какие действия выполняются, как объекты связаны.

- ▶ **Сущность** — значимый объект предметной области, сведения о котором должны быть представлены в информационной системе.
- ▶ **Атрибут** — характеристика сущности.
- ▶ **Связь** — отношение между сущностями предметной области.

Студент $\xrightarrow{\text{обучается в}}$ Группа

Студент $\xrightarrow{\text{получает}}$ Оценка

Концептуальная модель описывает предметную область **независимо от конкретной СУБД**.

На этом этапе неважно:

- ▶ PostgreSQL или Access;
- ▶ какой тип данных используется;
- ▶ каким SQL-запросом это будет реализовано.

Главное

Сначала понять предметную область, а затем выбирать техническую реализацию.

Определение

ER-модель (Entity–Relationship Model) была предложена Питером Ченом (Peter Chen) как концептуальная модель данных, предназначенная для представления семантики объектов предметной области и отношений между ними при проектировании баз данных.

Сущности + Атрибуты + Связи

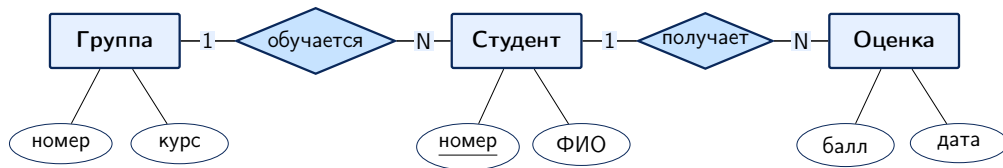
Графическое представление — **ER-диаграмма (ERD)**.

Базовые элементы ER-диаграммы (нотация Чена)

Элемент	Значение
Прямоугольник	сущность
Овал	атрибут
Ромб	связь
Линия	соединение элементов
Подчёркивание атрибута	первичный ключ

ER-диаграмма позволяет проще обсуждать структуру с заказчиком и находить ошибки до того, как система начала работать.

Пример ER-диаграммы (нотация Чена)



- ▶ прямоугольник — сущность, ромб — связь, овал — атрибут;
- ▶ подчёркивание атрибута — первичный ключ.

Более компактная нотация, часто используемая в практическом проектировании.



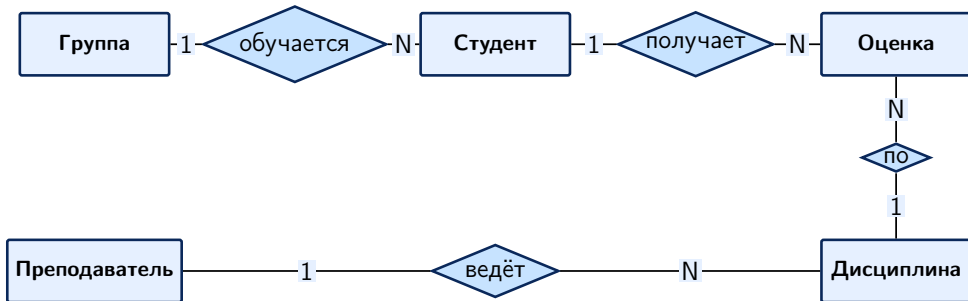
- ▶ двойная чёрточка — «ровно один»;
- ▶ окружность — необязательность связи («ноль»);
- ▶ «воронья лапка» — «много».

Типы связей (кардинальность)

Тип	Пример
1:1	Студент — Пропуск (паспорт — человек)
1:N	Группа — Студенты
N:M	Студенты — Дисциплины

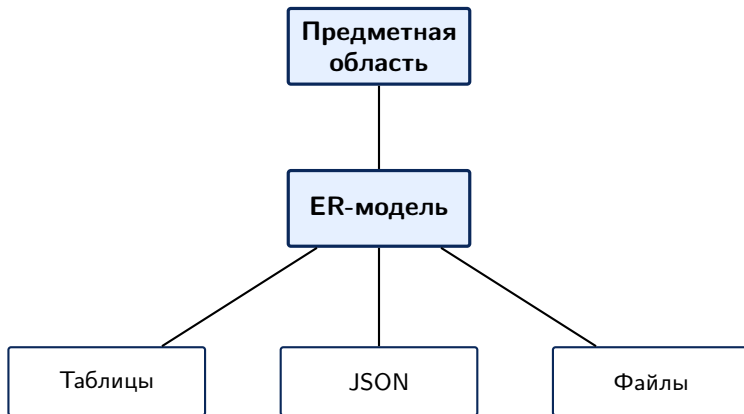
На практике особенно важна связь **многие-ко-многим** — она преобразуется в отдельную (связующую) таблицу.

Полная ER-диаграмма: предметная область «Университет»



Эта диаграмма — результат анализа предметной области и отправная точка для логического проектирования.

ER-модель и модель хранения — не одно и то же



Важно

ER-диаграмма не является таблицей и не определяет автоматически использование SQL или NoSQL — она описывает структуру и семантику предметной области

Трёхуровневая архитектура ANSI/SPARC

Это не этапы проектирования, а архитектурная модель самой СУБД.



Цель — независимость данных.

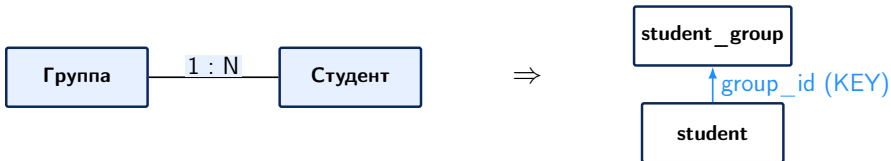
- ▶ **Внешний** — как данные видит конкретный пользователь или приложение (студент видит свои оценки, деканат — статистику факультета). В PostgreSQL этому соответствует VIEW.
- ▶ **Концептуальный** — единое логическое описание всей БД: сущности, атрибуты, связи, ограничения.
- ▶ **Внутренний** — как данные физически хранятся: файлы, страницы, индексы, буферы.

Главная идея

Пользователю не нужно знать, в каких физических страницах лежит строка таблицы.

Тип	Что позволяет
Физическая	изменять способы хранения и индексы без переписывания приложений
Логическая	изменять логическую структуру, сохраняя существующие представления

От ER-диаграммы к таблицам



```
CREATE TABLE student_group (  
    id SERIAL PRIMARY KEY,  
    name TEXT  
);
```

```
CREATE TABLE student (  
    id SERIAL PRIMARY KEY,  
    name TEXT,  
    group_id INTEGER REFERENCES student_group(id)  
);
```


Ограничения не позволяют БД перейти в некорректное состояние.

- ▶ PRIMARY KEY — уникальность и обязательность;
- ▶ FOREIGN KEY — ссылочная целостность;
- ▶ NOT NULL — обязательность значения;
- ▶ UNIQUE — уникальность значения;
- ▶ CHECK — произвольное условие на значение.

Идея

Правила должны контролироваться самой БД, а не только программой пользователя.

Нормализация (введение)

Нормализация — процесс организации данных, направленный на уменьшение избыточности и устранение аномалий.

Плохой вариант

student	name	discipline	teacher
1	Иванов	Математика	Сидоров
1	Иванов	Программирование	Козлов
2	Петров	Математика	Сидоров
2	Петров	Физика	Орлова

Проблема

Один и тот же факт (преподаватель, название дисциплины) повторяется много раз.

Изменение одного факта требует изменения множества строк → риск противоречий и аномалий.

Нормализованные таблицы

student	
id	name
1	Иванов
2	Петров

discipline	
id	title
1	Математика
2	Программирование
3	Физика

grade		
student_id	disc_id	score
1	1	5
1	2	5
2	1	4
2	3	3

Преимущества

- ▶ Нет избыточности
- ▶ Изменение одного факта — в одном месте
- ▶ Легче поддерживать целостность
- ▶ Уменьшается риск аномалий

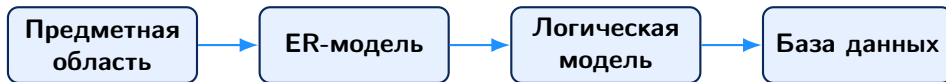
Важно

Выделяют следующие нормальные формы: 1НФ, 2НФ, 3НФ и т.д.

SQL объединяет различные средства работы с реляционными БД (DDL, DML и другие). SQL не заменяет проектирование.

	Access	PostgreSQL
Тип	настольная СУБД	серверная СУБД
Интерфейс	сильный GUI	SQL + инструменты
Масштаб	небольшие системы	крупные системы

- ▶ **draw.io / diagrams.net** — универсальный инструмент построения ER-диаграмм;
- ▶ **pgModeler** — моделирование PostgreSQL;
- ▶ **DBeaver** — работа с различными СУБД и ER-диаграммами;
- ▶ **pgAdmin** — графическое администрирование PostgreSQL;
- ▶ **psql** — консольный клиент PostgreSQL (CLI).



Главная идея

Сначала мы понимаем что нужно хранить, затем строим модель и только после этого реализуем базу данных.

- ▶ Выбор способа хранения — **результат** анализа данных, а не исходная точка проектирования.
- ▶ Табличное, документное и файловое представления не исключают друг друга и могут сочетаться в одной системе.
- ▶ ER-модель — семантическое описание предметной области, не привязанное к конкретной технологии.
- ▶ Разделение уровней (внешний / концептуальный / внутренний) обеспечивает независимость данных и гибкость системы при её росте.

Спасибо за внимание!